Strongly Solving Okiya: Complete Game-Theoretic Analysis and Statistical Win Dynamics via GPU-Accelerated Brute Force

Aleksei Iurasov

Abstract

We present the first strong solution of Okiya (Niya), a two-player combinatorial board game on a 4×4 grid with attribute-constrained tile placement and 19 winning patterns. Using a GPU-accelerated forward-BFS/backward-minimax solver on an NVIDIA RTX 4090, a single board configuration is resolved in approximately 20 s, enumerating $\sim 9.3 \times 10^6$ reachable states—8.7% of the theoretical upper bound. We solve 1,000 uniformly random configurations (seed 42, ∼1 h total) and find that the first player (P0) wins 67.2% of configurations with zero draws, yet the mean number of winning opening moves is only 1.37 out of 12, revealing a structural but *fragile* first-mover advantage. A per-level win-probability gradient $f(L)$ exhibits a pronounced turn-parity sawtooth with shrinking amplitude, indicating that the game's outcome crystallises progressively rather than at a single critical level. Across all configurations, 46.1% of terminal states are decided by the stuck rule rather than pattern completion, highlighting its decisive role in Okiya's dynamics.

1 Introduction

1.1 Motivation

Combinatorial game theory [13] has produced landmark results ranging from the exhaustive analysis of Tic-Tac-Toe to the 18-year computation that solved Checkers [8]. The theoretical foundations—from Zermelo's determinacy theorem for chess [1] to von Neumann's minimax theorem [2]—guarantee that finite two-player zero-sum games possess optimal strategies, but *finding* those strategies requires complete enumeration or sophisticated search. Between the trivially solvable and the computationally intractable lies a gap: games that are too complex for pencil-and-paper analysis yet small enough for brute-force computation on modern hardware [14]. Filling this gap is scientifically valuable because it provides exact benchmarks for heuristic search, validates game-theoretic predictions, and reveals structural phenomena invisible to partial analyses.

Okiya—originally published as Niya (2012) and rebranded by Blue Orange Games as Okiya (2013) [11]—is a two-player abstract strategy game designed by Bruno Cathala. Played on a 4×4 grid with attribute-constrained placement and dual win conditions (pattern completion and opponent stuck), its state space of $\sim 9.5 \times 10^6$ reachable positions makes it an ideal candidate for exact solution. The game is nontrivial—the relation constraint between successive moves creates a complex DAG rather than a tree, and the stuck rule introduces a second strategic axis absent from pure pattern-matching games.

In this work we provide a *strong solution* of individual Okiya configurations: for each solved configuration, the minimax value is computed for every reachable state, enabling perfect play from any position. Because each of the $\sim 4.5 \times 10^9$ distinct configurations defines a different game DAG, a strong solution of the full game would require solving all of them. We solve 1,000 random configurations to characterise the game's statistical landscape.

Table 1: Comparison of solved two-player games (ordered by year solved). State-space sizes are approximate and refer to reachable positions.

Game	States	Year	Method	Outcome	Ref.
Tic-Tac-Toe	5.5×10^{3}	1952	Exhaustive	Draw	folklore
Connect Four	4.5×10^{12}	1988	Alpha-beta	P1 wins	[5]
Nine Men's Morris	1.0×10^{10}	1996	Retrograde	Draw	[6]
Checkers	5.0×10^{20}	2007	Retrograde	Draw	[8]
Pentago	3.0×10^{15}	2014	Alpha-beta+DB	P1 wins	[12]
Okiya	$\mathbf{9.5\times10^{6}}^{\dagger}$	**2026**	**GPU BFS+minimax**	**P0 67.2%***	**This work**

*Configuration-dependent; P0 wins 67.2% of 1,000 random configs (95% CI: [64.3%, 70.1%]).

†Per configuration; $\sim 4.5 \times 10^{9}$ distinct configurations exist.

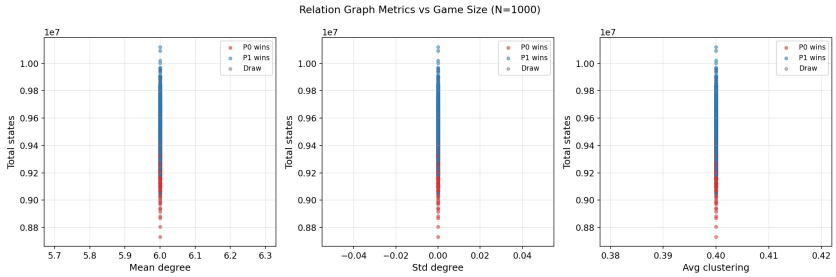

Figure 1: Tile-relation graph. Each node is one of the 16 tile types (plant×weather); an edge connects two tiles that share at least one attribute. Every tile has exactly 6 neighbours (degree 6, clustering coefficient 0.4), and this structure is invariant across all configurations up to relabelling.

1.2 Related work

Table 1 places Okiya in the context of previously solved combinatorial games. Retrograde analysis [4] and alpha-beta search [3] have been the dominant techniques for such solutions. With $\sim 9.5\times10^{6}$ reachable states per configuration, Okiya sits between Tic-Tac-Toe and Nine Men's Morris in complexity, occupying a niche where GPU brute force is both sufficient and informative.

2 Game Rules and Formalization

2.1 Board, tiles, and attributes

Okiya is played on a 4×4 grid of 16 positions. Before the game begins, each position is assigned a unique tile bearing two orthogonal attributes drawn from:

- **Plant** (4 types): Maple, Cherry, Pine, Iris.
- **Weather** (4 types): Sun, Tanzaku, Bird, Rain.

The $4 \times 4 = 16$ attribute combinations yield 16 distinct tile types, one per position (Figure 1). A *board configuration* is a permutation of these 16 tile types on the 16 positions; there are 16! possible configurations.

2.2 Move constraints and win conditions

Two players, P0 (Red, first mover) and P1 (Blue), alternate turns. On each turn a player claims an unclaimed tile subject to the following constraints:

1. **Opening move (turn 0):** P0 must select one of the 12 *border* positions (all except the four interior squares at positions 5, 6, 9, 10 in row-major order).
2. **Subsequent moves:** The current player must select an unclaimed tile that is *related* to the last-played tile. Two tiles are related if they share at least one attribute (same plant *or* same weather).

A player wins by either:

- **Pattern completion:** claiming four tiles that form one of 19 winning patterns (4 rows, 4 columns, 2 diagonals, 9 overlapping 2×2 blocks); or
- **Stuck rule:** leaving the opponent with no valid move—the stuck player loses immediately.

2.3 State encoding

Each game state is packed into a single `uint64` using 37 bits:

$$s = (mask_{P0} \ll 21) \mid (mask_{P1} \ll 5) \mid last\_pos \tag{1}$$

where $mask_{P0}$ and $mask_{P1}$ are 16-bit bitmasks of each player's claimed positions, and $last\_pos \in \{0, \ldots, 16\}$ encodes the most recently played position (16 denotes the initial state with no previous move). This encoding is injective, sortable (enabling $O(\log n)$ binary-search lookups), and compact (8 bytes per state).

The *level* of a state is $L = \text{popcount}(mask_{P0}) + \text{popcount}(mask_{P1})$, the total number of tiles placed. At level L, P0 has placed $\lceil L/2 \rceil$ tiles and it is P0's turn if L is even.

2.4 Configuration space

The tile arrangement is fixed before play. There are $16! \approx 2.09 \times 10^{13}$ possible configurations. Symmetry reductions include:

- Board symmetry (dihedral group D_4, order 8): rotations and reflections.
- Plant relabelling ($4! = 24$ permutations).
- Weather relabelling ($4! = 24$ permutations).

Under these reductions, the number of *distinct* configurations is

$$\frac{16!}{8 \times 24 \times 24} \approx 4.5 \times 10^9.$$

Each distinct configuration produces a different game DAG with different state counts, game value, and winning moves.

3 GPU Solver Architecture

3.1 Two-phase overview

The solver employs a classical *forward enumeration + backward induction* strategy:

1. **Phase 1 — Forward BFS:** Starting from the single root state (empty board, L=0), expand all reachable states level by level up to L=16.
2. **Phase 2 — Backward minimax:** Starting from the deepest level, propagate game-theoretic values upward to the root.

This produces a *strong solution*: the minimax value and an optimal strategy are known for every reachable state, not merely the root.

Algorithm 1 Forward BFS (state enumeration)

$\mathcal{S}_0 \leftarrow \{s_{\text{root}}\}$
for $L = 0$ **to** 15 **do** ▷ generates $\mathcal{S}_{L+1}$ up to $\mathcal{S}_{16}$
  GPU: expand non-terminal states in $\mathcal{S}_L$ ▷ $16n$ threads
  for each parent $s \in \mathcal{S}_L$, candidate position $p \in \{0, \ldots, 15\}$ **do**
    if p unclaimed **and** move valid **then**
      $s' \leftarrow \textsc{ApplyMove}(s, p)$
      if mover wins **then** mark s' terminal with value ± 1
      end if
      emit s'
    end if
  end for
  CPU: filter sentinels, deduplicate, detect stuck states
  $\mathcal{S}_{L+1} \leftarrow$ unique children
  Build CSR edges from $\mathcal{S}_L$ to $\mathcal{S}_{L+1}$
end for

Algorithm 2 Backward minimax (value propagation)

for $L = L_{\max} - 1$ **downto** 0 **do**
  for each non-terminal state $s \in \mathcal{S}_L$ **do** ▷ GPU, n threads
    $C \leftarrow$ children of s via CSR
    if L even (P0's turn) **then**
      $v(s) \leftarrow \max_{c \in C} v(c)$ ▷ exit early on $+1$
    else
      $v(s) \leftarrow \min_{c \in C} v(c)$ ▷ exit early on -1
    end if
  end for
end for

3.2 Forward BFS

Algorithm 1 summarises the forward phase. At each level, a CUDA kernel generates all candidate children in parallel (one thread per parent–position pair, $16n$ threads for n parents), writing valid children to global memory with a sentinel value (`0xFFFF...F`) for invalid entries. The CPU then filters sentinels, deduplicates via `np.unique`, detects stuck (no-move) states, and builds a Compressed Sparse Row (CSR) parent-to-child edge structure for the subsequent minimax phase.

Move validity at level L=0 requires a border position ($pos \,\&\, 0\text{xF99F} \neq 0$); at L>0 it requires the candidate position to share an attribute with the last-played tile, checked via a precomputed 16-entry relation-mask table in CUDA constant memory. Win detection compares the mover's bitmask against all 19 winning patterns (Appendix A).

Deduplication is essential because the game graph is a *DAG*, not a tree: the same state can be reached by different move orders. After `np.unique`, the sorted state array supports $O(\log n)$ child-index lookups via `np.searchsorted`.

3.3 Backward minimax

Algorithm 2 propagates values from terminal states upward using the classical minimax principle [3]. Values are stored as `int8`: $+1$ (P0 wins), -1 (P1 wins).

Because the value domain is binary ($\{+1, -1\}$, with no draws—see Section 8.2), the early-exit optimisation is especially effective: a maximising node can stop on the first $+1$ child and a

Table 2: GPU memory profile at peak level (L=12, ∼2.1 M parent states).

Data structure	Per state	Peak total
States (`uint64`)	8 B	17 MB
Values (`int8`)	1 B	2 MB
Terminal flags (`uint8`)	1 B	2 MB
CSR offsets (`int32`)	4 B	9 MB
CSR indices (`int32`)	var.	∼12 MB
Expansion buffer (16×8 B)	128 B	∼282 MB
Constant memory (win patterns, relation masks, border)		144 B

Table 3: Solver timing breakdown (single configuration, RTX 4090).

Phase	Time
CUDA compilation (cached)	∼1 s
Forward BFS (GPU + CPU dedup)	∼8 s
Backward minimax (GPU)	∼5 s
Verification (CPU sampling)	∼5 s
File I/O (compressed npz)	∼1 s
Total	**∼20 s**

minimising node can stop on the first −1 child.

3.4 CUDA implementation details

Table 2 summarises the GPU resource profile. The solver is implemented in PyCUDA [10], following the GPU-based state-space exploration paradigm of Edelkamp and Sulewski [9]. All layout-dependent data (19 win-pattern bitmasks, 16 relation masks, 1 border mask) fits in 144 bytes of CUDA constant memory, residing in L1 cache for the duration of both kernels. Both kernels use a block size of 256 threads.

Peak GPU memory is dominated by the expansion buffer at levels 11–12, where each of ∼2 M parent states generates up to 16 candidate children (∼35 M slots, ∼282 MB). The CPU-side bottleneck is `np.unique` on arrays of up to ∼35 M `uint64` entries.

Table 3 shows the end-to-end timing for a single configuration on an NVIDIA RTX 4090. The forward BFS takes approximately 8 s (GPU expansion plus CPU deduplication) and backward minimax takes approximately 5 s. For the 1,000-configuration batch run, CUDA compilation is amortised and verification is abbreviated, yielding ∼3.5 s per configuration.

3.5 Verification

Three independent checks validate the solution:

1. **Structural invariants** (exhaustive): player masks are disjoint, popcount matches level, last position belongs to the correct player's mask.

2. **Minimax consistency** (sampled, up to 50,000 states per level for interactive runs; exhaustive for the canonical board): each non-terminal value equals the max (P0 turn) or min (P1 turn) of its children's values via CSR lookup. At ∼9.3 M total states, exhaustive verification is computationally inexpensive (∼2 s) and was performed for the canonical configuration.

Table 4: Theoretical upper bound vs. reachable states by level (canonical board).

Level	Upper bound	Reachable	%	Mover
0	1	1	100.0	P0
1	16	12	75.0	P1
2	480	72	15.0	P0
3	5,040	360	7.1	P1
4	43,680	1,524	3.5	P0
5	218,400	6,866	3.1	P1
6	960,960	24,198	2.5	P0
7	2,802,800	90,332	3.2	P1
8	7,207,200	244,503	3.4	P0
9	12,972,960	671,170	5.2	P1
10	20,180,160	1,227,894	6.1	P0
11	22,198,176	2,101,611	9.5	P1
12	20,180,160	2,122,834	10.5	P0
13	12,492,480	1,772,951	14.2	P1
14	5,765,760	779,608	13.5	P0
15	1,544,400	229,834	14.9	P1
16	205,920	23,976	11.6	P0
Total	**106,778,593**	**9,297,746**	**8.7**	

3. **State-count comparison**: per-level counts are printed for manual inspection against reference values.

4 Canonical Board Results

We first report detailed results for a single *canonical* configuration (tile permutation $[14, 4, 11, 9, 7, 12, 10, 3, 8, 1, 5$ in row-major order, fixed at project inception) before generalising to 1,000 random configurations in Sections 5–6.

4.1 Game value

The canonical board has game value $v(s_{\text{root}}) = +1$: P0 wins with perfect play. However, the advantage is extremely fragile: only 1 out of 12 legal opening moves leads to a P0 win. The remaining 11 opening moves all result in P1 victories under optimal defence. At level 1, the fraction of P0-winning states drops to $f(1) = 1/12 \approx 0.083$, the sharpest single-move swing in the entire game.

4.2 State space

The canonical board contains 9,297,746 reachable states, approximately 8.7% of the theoretical upper bound of ∼107 M states derived from combinatorial counting (ignoring the relation constraint). The per-level state counts peak at level 12 with 2,122,834 states (22.8% of the total).

Table 4 shows the theoretical upper bound and observed reachable states by level. The relation constraint prunes approximately 91% of theoretically possible states, with pruning most severe at early levels and relaxing toward the endgame.

4.3 Terminal state classification

Of the 9,297,746 reachable states, 1,938,231 (20.8%) are terminal. These divide into two categories:

- **Pattern wins:** 1,083,641 states (55.9% of terminals) where a player completed one of the 19 winning patterns.
- **Stuck wins:** 854,590 states (44.1% of terminals) where the active player has no legal move and therefore loses.

The stuck rule is nearly as impactful as pattern completion in determining game outcomes. The earliest terminal states appear at level 7 (P0's fourth tile, the minimum for a four-in-a-pattern win); stuck states first appear at level 8. The mean game length is 12.66 moves, with the mode at level 13 (30.0% of terminal states).

4.4 Win rate under random play

Beyond minimax values, we compute the probability of a P0 win under *uniform random play* (each player selects uniformly among valid moves) via bottom-up aggregation over the full DAG. From the root, P0's win probability under random play is **51.9%**—a modest advantage compared to the decisive +1 under optimal play. This divergence illustrates that many of P1's minimax-losing positions require precise defensive play to exploit, making P0's advantage practically less dominant than the game value suggests.

5 Statistical Analysis: 1,000 Configurations

5.1 Methodology

We sample 1,000 board configurations by permuting the 16 tile types using a seeded pseudo-random generator (seed 42, NumPy default RNG). These configurations are sampled from the raw 16! permutation space without symmetry deduplication; given the $\sim 4.5 \times 10^9$ equivalence classes, the probability of any two samples being symmetry-equivalent is negligible ($< 10^{-4}$ by the birthday bound). Each configuration is solved end-to-end (BFS + minimax) with per-level metrics extracted in-memory. The total computation takes approximately 1 hour on a single RTX 4090 ($\sim$3.5 s per configuration), with CUDA compilation amortised across runs.

5.2 Outcome distribution

Figure 2 shows the outcome distribution. Of 1,000 configurations:

- **672 (67.2%, 95% Wald CI: [64.3%, 70.1%])** are won by P0 (first player).
- **328 (32.8%)** are won by P1 (second player).
- **0 (0.0%)** end in draws.

The absence of draws across $\sim 9.48 \times 10^9$ aggregate reachable states is consistent with the structural draw impossibility proven in Theorem 1 (Section 8.2).

5.3 State-space variation

The total number of reachable states is remarkably stable across configurations: mean 9,481,800, standard deviation 204,000 (2.2% coefficient of variation), range 8,731,257–10,120,199. This near-invariance stems from the uniform relation graph: every tile is related to exactly 6 others (degree 6.0, clustering coefficient 0.4, identical across all configurations up to relabelling). The

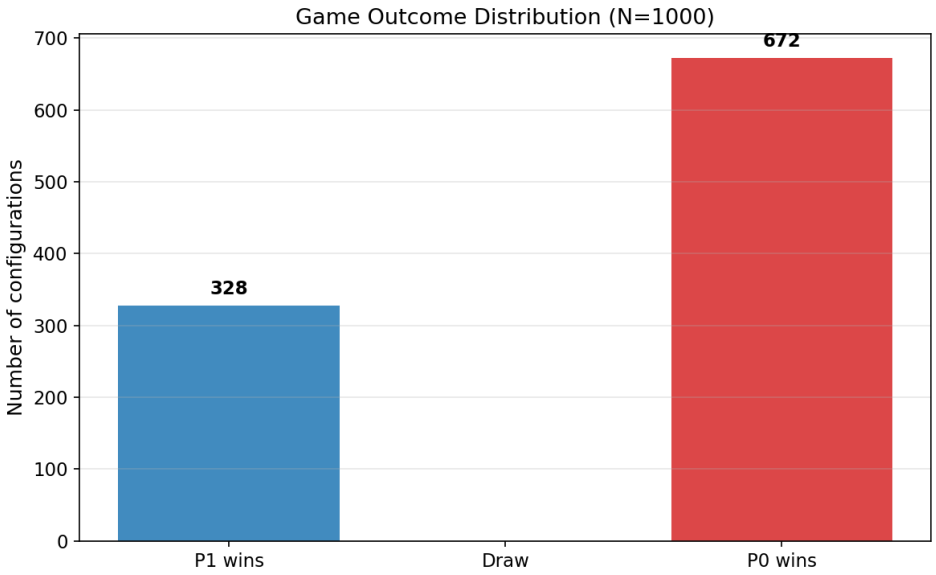

Figure 2: Outcome distribution across 1,000 random configurations. P0 wins 672 configurations (67.2%), P1 wins 328 (32.8%), with zero draws.

combinatorial structure of the game graph is thus dominated by the attribute-relation topology rather than the specific tile placement.

5.4 Winning openings

Figure 3 shows the distribution of winning opening moves (out of 12 border positions) for P0. The mean is 1.37 (95% CI: [1.26, 1.48]) with a median of 1 and a mode of 0.

- 328 configurations (32.8%) have *zero* winning openings (P1 wins).
- 292 (29.2%) have exactly 1 winning opening.
- 196 (19.6%) have 2 winning openings.
- 184 (18.4%) have 3–8 winning openings.

Even when P0 wins, the advantage is narrow: nearly half of P0-winning configurations (292 of 672, 43.5%) offer only a single correct first move.

6 Win Dynamics

6.1 Win-probability gradient

For each configuration, define the *P0-win fraction* at level L as

$$f(L) = \frac{|\{s \in \mathcal{S}_L : v(s) = +1\}|}{|\mathcal{S}_L|}, \tag{2}$$

the fraction of reachable states at level L whose minimax value is a P0 win. Since Okiya is zero-sum with no draws, the P1-win fraction is $g(L) = 1 - f(L)$.

Figure 4 plots the mean $f(L)$ across 1,000 configurations with ± 1 standard deviation bands. The curve does not rise monotonically; instead it exhibits a pronounced *sawtooth* superimposed on a gradually converging trend. The confidence bands are tight (std < 0.05 for $L \geq 6$), confirming that the sawtooth is a universal property of Okiya, not an artefact of specific configurations.

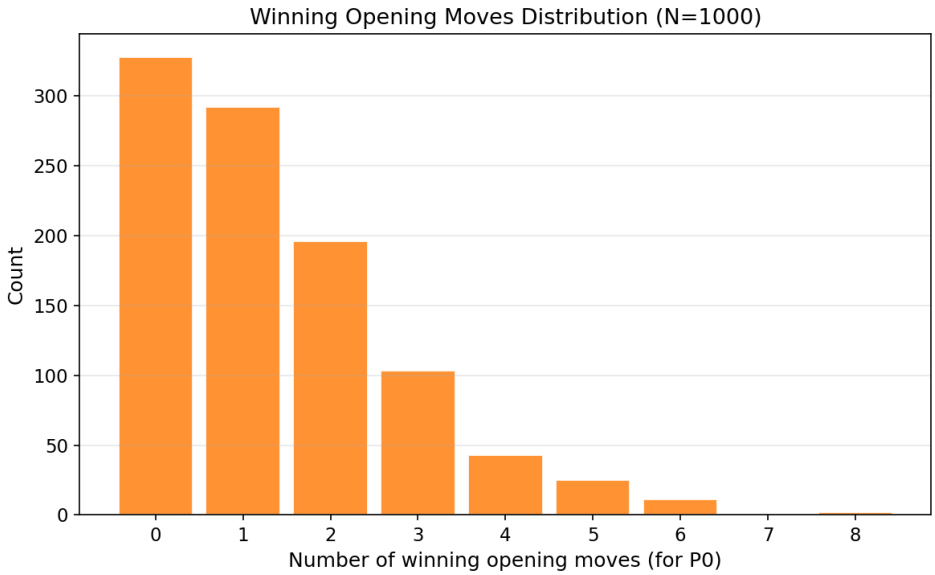

Figure 3: Histogram of the number of winning opening moves for P0 across 1,000 configurations. The mode is 0 (P1-winning configurations); among P0 wins, a single winning opening is most common.

6.2 Turn-parity sawtooth

The sawtooth arises from the coupling of level and turn. At even levels, P0 moves and can steer toward favourable positions (f increases); at odd levels, P1 moves and pushes back (f decreases). Figure 5 isolates this effect by plotting the first difference $\Delta f(L) = f(L) - f(L-1)$ as a bar chart with error bars across configurations.

The sawtooth amplitude is largest at the opening ($|\Delta f(1)| \approx 0.56$, reflecting the drop from $f(0) = 0.672$—which equals the aggregate P0-win rate, since level 0 contains exactly one state per configuration—to $f(1) = 0.11$) and decreases toward mid-game ($|\Delta f| \approx 0.2$–0.3 at levels 8–12), before a final spike at levels 15–16 driven by the stuck rule endgame. The shrinking amplitude indicates that individual moves carry less leverage as the game progresses—the outcome is increasingly determined by the accumulated position rather than any single move.

6.3 Critical move ratio

For each non-terminal state, we check whether any child has a different minimax value than the parent. The *critical move ratio* (CMR) at level L is the fraction of non-terminal states at that level where at least one such value-changing child exists—i.e., where the current player's choice is decisive.

On the canonical board, $\mathrm{CMR}(0) = 0.92$: eleven of twelve opening moves change the game's outcome. By mid-game ($L \approx 6$–10), CMR ≈ 0.3, and by the endgame ($L \geq 14$), CMR $\to 0$—the outcome is already determined regardless of remaining moves. Figure 6 shows this monotonic decline across all 1,000 configurations.

6.4 Stuck-state fraction

The fraction of terminal states decided by the stuck rule (rather than pattern completion) grows monotonically from 0% at level 7 to 76% at level 16 across the 1,000-configuration aggregate. Early terminations ($L \leq 9$) are dominated by pattern wins, while late terminations ($L \geq 14$) are overwhelmingly stuck outcomes. Overall, 46.1% of all terminal states (across $\sim 1.93 \times 10^9$ aggregate terminals) are stuck wins. The per-configuration mean stuck fraction is 46.1% with

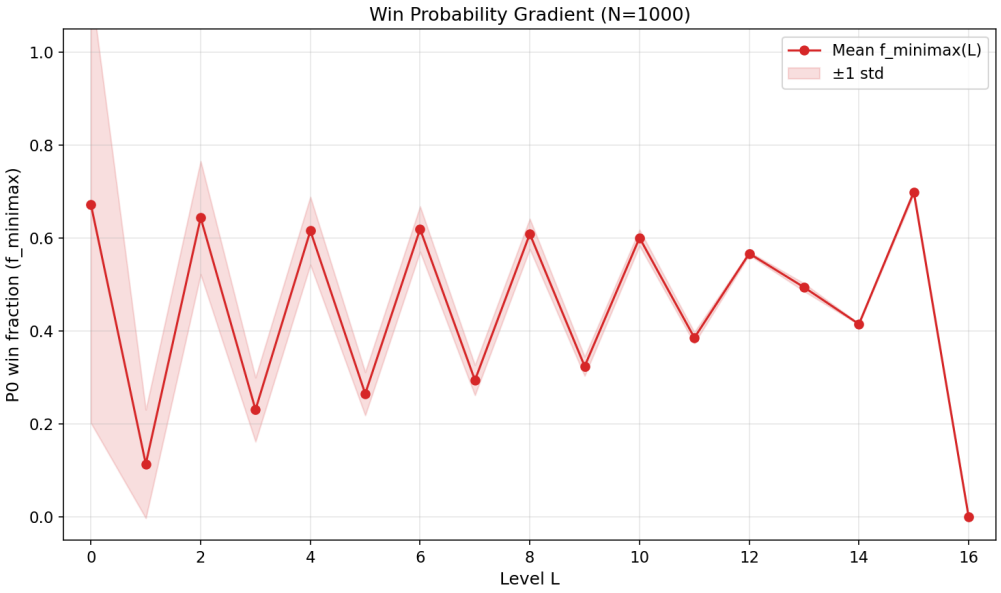

Figure 4: P0-win fraction $f(L)$ across game levels, averaged over 1,000 configurations (± 1 std shaded). The sawtooth pattern reflects turn-parity coupling: $f(L)$ rises on even levels (P0's turn) and falls on odd levels (P1's turn).

a 95% CI of [45.3%, 46.9%] over the 1,000 configurations, indicating low cross-configuration variance. The stuck rule is thus a co-equal determinant of Okiya's outcome. Figure 7 illustrates this level-by-level transition from pattern-dominated to stuck-dominated terminations.

7 Heuristic Distillation

A strong solution is a lookup table with millions of entries—useful for theoretical analysis but impractical as a cognitive strategy. A natural follow-up question is: how much of the minimax solution can be *distilled* into concise, human-playable rules? We address this by extracting interpretable heuristics directly from the labeled game DAG of the canonical configuration and evaluating them against the oracle.

7.1 Feature engineering and mutual information

We define 11 features per (state, candidate move) pair, spanning four categories: *offensive* (pattern completion, threat count), *defensive* (opponent pattern progress, block urgency), *constraint* (opponent mobility after the move, stuck threat), and *positional* (pattern participation count, game level).

To identify which features matter at each game phase, we compute the mutual information $\mathrm{MI}(\mathrm{feature}, \mathrm{is\_optimal})$ stratified by level on a sample of $\sim$23,000 informative states (those with both optimal and suboptimal moves, yielding $\sim$80,000 labeled move pairs). Table 5 summarises the results.

The phase transition from positional features to constraint features independently confirms the structural observation from Section 6: Okiya's strategy operates on two axes (pattern completion and stuck threat), with the balance shifting from the former to the latter as the game progresses.

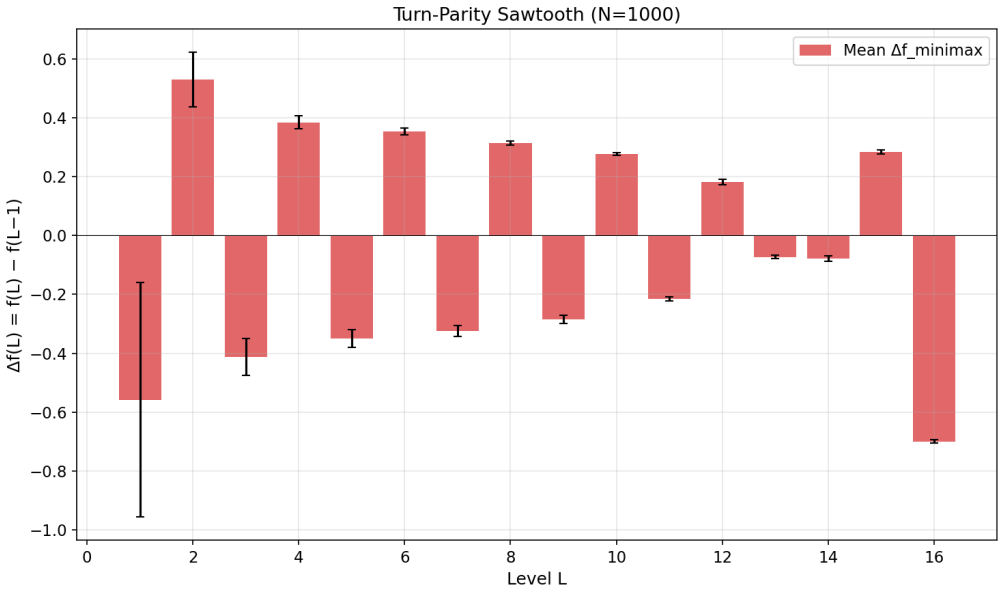

Figure 5: First difference $\Delta f(L) = f(L) - f(L{-}1)$ of the P0-win fraction. Positive bars (even levels) indicate P0's turn improving its position; negative bars (odd levels) indicate P1's countermoves. Error bars show ± 1 std across 1,000 configs.

Table 5: Dominant features by game phase (mutual information with optimal-move label). The strategic axis shifts from positional play (early) through pattern building (mid) to constraint play (late).

Level range	Top features (MI)	Interpretation
L=0	None (MI=0 for all)	Opening is a memorisation problem
L=1–3	Pattern participation (0.25)	Positional value dominates
L=4–6	Own best pattern (0.07)	Pattern building
L=7–9	Opponent mobility (0.08)	Constraint play emerges
L=10–12	Own best pattern (0.18), opp. mobility (0.17), win-now (0.16)	Three-way race: win, constrain, or lose
L=13–14	Win immediately (0.69)	Near-deterministic endgame

7.2 Heuristic hierarchy and move accuracy

We evaluate a family of heuristics ranging from trivial to learned, measuring *move accuracy*: the fraction of informative states where the heuristic selects a minimax-optimal move. Table 6 presents the results.

A four-rule heuristic (*win if you can; block if you must; build on high-value positions; constrain the opponent*) closes roughly one-third of the gap between random play and perfection. A shallow decision tree (depth 4) closes 56% of the gap, discovering level-dependent thresholds that hand-crafted rules miss.

The depth-4 tree yields five human-readable rules:

1. If you can complete a win pattern, play it.
2. If the opponent will be stuck (0 valid moves), play it.
3. If the opponent will have only 1 valid move, play it.

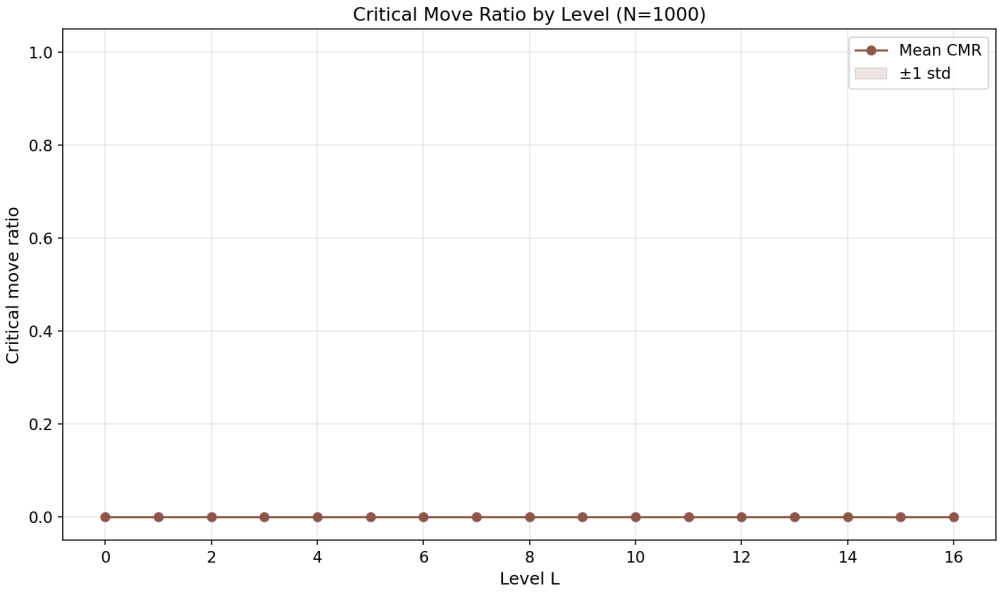

Figure 6: Critical move ratio (CMR) by level, averaged over 1,000 configurations (±1 std shaded). The ratio declines from ∼0.9 at the opening to near zero by level 14, quantifying how early the game's outcome crystallises.

4. At high-value positions (≥6 pattern participations): play if the opponent has no imminent 3-in-a-pattern threat; otherwise block.

5. Otherwise, avoid the move.

In natural language: *Win if you can. Trap if you can't win. Build on valuable positions, but watch for threats.*

7.3 The opening bottleneck

A striking result is that *every* heuristic achieves 0% accuracy at L=0. On the canonical board, all 12 border positions produce identical feature vectors: each participates in exactly 4 win patterns, each has 6 related tiles, and no blocking or stuck threats exist on an empty board. The winning opening depends on the *global* structure of the tile-relation graph—which positions create advantageous constraint cascades 5–10 moves later—not on any locally computable property.

This confirms quantitatively that the "fragile first-mover advantage" identified in Sections 5.4 and 6 is concentrated in board-specific global structure rather than locally computable features. A practical Okiya strategy would therefore combine *opening preparation* (memorise the winning first move for the given board) with *heuristic play* from L=1 onward—paralleling chess, where opening theory is memorised and heuristic evaluation governs the middle game.

7.4 Limitations of the heuristic analysis

The heuristic evaluation is conducted on the canonical board configuration only. While the near-invariant state-space size across 1,000 configurations (2.2% CV, Section 5) suggests that the MI profile and accuracy numbers should generalise, this remains to be verified empirically. Multi-configuration heuristic evaluation is left for future work.

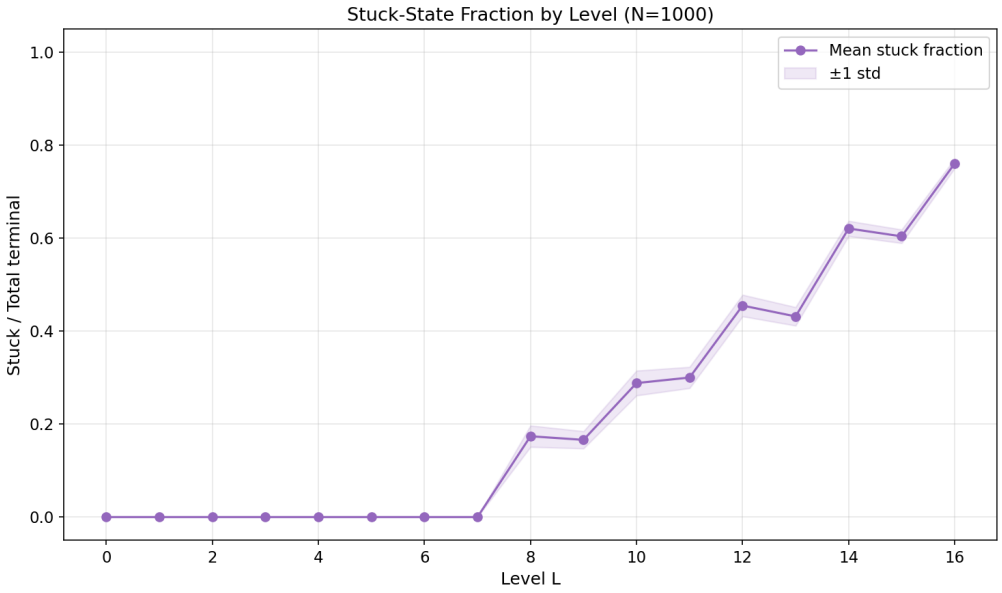

Figure 7: Fraction of terminal states decided by the stuck rule (vs. pattern completion) by level, aggregated over 1,000 configurations. Stuck outcomes dominate from level 14 onward, reaching 76% at level 16.

8 Discussion

8.1 Classification

Following the taxonomy of van den Herik et al. [7], our result constitutes a *strong solution* of Okiya: the minimax value is computed for every reachable state of each solved configuration. In complexity, Okiya sits between Tic-Tac-Toe (5.5×10^3 states, trivially solved) and Nine Men's Morris (10^{10} states, requiring retrograde analysis [6]). The GPU brute-force approach is both sufficient and informative at this scale.

8.2 Draw impossibility

We prove that draws are impossible in Okiya.

Theorem 1. *Every reachable Okiya state has minimax value $v(s) \in \{+1, -1\}$. No draw ($v(s) = 0$) can occur under any configuration.*

Proof. We proceed by backward induction on the game DAG.

Base case (terminal states). A state s is terminal if either (a) the current player's move completes one of the 19 winning patterns, in which case $v(s) = +1$ or -1 depending on the winner; or (b) the current player has no legal move (stuck), in which case that player loses, giving $v(s) \in \{+1, -1\}$. There is no third terminal condition: at level 16, either the 16th tile itself completes a winning pattern (a pattern win at the final move) or no unclaimed tiles remain and the current player is trivially stuck. In both cases the outcome is decisive. Thus every terminal state has value in $\{+1, -1\}$.

Inductive step (non-terminal states). Let s be a non-terminal state at level L. By minimax, $v(s) = \max_{c \in C} v(c)$ if it is P0's turn, or $v(s) = \min_{c \in C} v(c)$ if it is P1's turn, where C is the set of children of s. By the inductive hypothesis, $v(c) \in \{+1, -1\}$ for all $c \in C$. Since $\max\{+1, -1\} = +1$, $\min\{+1, -1\} = -1$, $\max\{+1\} = +1$, $\min\{-1\} = -1$, etc., the result $v(s) \in \{+1, -1\}$ follows in all cases. □

Table 6: Heuristic hierarchy: move accuracy (agreement with minimax oracle) and win rate against a uniform-random opponent (500 games, averaged over both roles). Canonical configuration.

ID	Heuristic	Accuracy	Win rate
H0	Random	0.41	50.0%
H1	Win-or-block (2 rules)	0.52	65.6%
H2	+ Minimise opp. mobility (3 rules)	0.62	74.1%
H3	+ Maximise own patterns (4 rules)	0.65	82.6%
H4	Decision tree, depth 4 (∼5 splits)	0.74	83.7%
H5	Decision tree, depth 8 (∼30 splits)	0.78	84.5%
Oracle	Minimax lookup	1.00	99.8%‡

‡The Oracle loses when playing as the losing side (P1 on a P0-winning board); even perfect play cannot overcome a lost position.

This structural property is confirmed empirically: across 1,000 configurations and $\sim 9.48 \times 10^9$ aggregate reachable states, zero draws were observed.

8.3 First-mover advantage: structural but fragile

P0 wins 67.2% of random configurations—a significant first-mover advantage. Yet the advantage is *fragile*: the mean number of winning opening moves is only 1.37 out of 12, and 43.5% of P0-winning configurations offer only a single correct opening. Under uniform random play (51.9% P0 win rate on the canonical board), the advantage nearly vanishes, indicating that it requires precise play to realise.

The fragility is captured by the P0-win fraction at level 1: $f(1) = 0.11$ averaged across configurations, meaning that after P0's first move, only 11% of resulting positions are P0-winning. The "advantage" is better described as *access to a narrow winning path* rather than a broad strategic superiority.

8.4 Generalizability

The two-phase solver (forward BFS + backward minimax on a DAG) is applicable to any game satisfying:

1. **Bounded state space:** all reachable states fit in GPU memory ($\sim 10^7$–10^8 states with current hardware).
2. **DAG structure:** the game graph is acyclic, enabling level-by-level processing.
3. **Sortable encoding:** states can be packed into fixed-width integers for efficient deduplication and lookup.

Candidate games include Quarto variants (attribute-based placement on a grid), modified placement games with relation constraints, and $N \times N$ generalisations of Okiya.

8.5 Strategic compressibility

The heuristic analysis in Section 7 suggests a preliminary observation about game complexity. Define the *heuristic ceiling* $\kappa(k)$ as the maximum move accuracy achievable by a depth-k decision tree trained on the complete solution: $\kappa(k) = \max_{T \in \mathcal{T}_k} \mathrm{Acc}(T)$, where $\mathcal{T}_k$ is the set of all depth-k trees over the feature space. For Okiya (canonical configuration), $\kappa(4) = 0.74$ and $\kappa(8) = 0.78$, with an estimated asymptote of ∼0.85–0.90. The ∼15–20% residual gap represents decisions

that require deep lookahead or board-specific global knowledge and resist compression into local features. Two games with similar state-space sizes might exhibit very different heuristic ceilings, indicating different *strategic* complexities. We note this as a potentially useful complement to state-space size, though validation across multiple games and configurations is needed before it can be considered a robust complexity measure.

8.6 Limitations

This work has several limitations. First, only 1,000 of $\sim 4.5 \times 10^9$ distinct configurations are solved; while the low variance in state-space size and the confidence intervals on the P0-win rate suggest the sample is representative, configuration-level features that predict whether a board is P0- or P1-winning remain unexplored. Second, the heuristic analysis (Section 7) is evaluated on the canonical configuration only; multi-configuration validation is needed. Third, the solver is benchmarked on a single GPU (NVIDIA RTX 4090, 24 GB VRAM); performance on smaller GPUs is not characterised, though peak memory usage ($\sim$282 MB) suggests broad compatibility. Fourth, the 1,000-configuration batch uses sampled (not exhaustive) minimax verification; exhaustive verification was performed only for the canonical configuration. Finally, we do not compare against alternative solving strategies (e.g., depth-first alpha-beta), which may be faster for determining the root value alone without computing all state values.

9 Conclusion and Open Questions

Contributions. This work makes four contributions:

1. **Strong solution of individual Okiya configurations:** the first complete game-theoretic analysis, computing minimax values for all $\sim 9.3 \times 10^6$ reachable states per configuration in $\sim$20 s on an RTX 4090.
2. **Statistical landscape:** a 1,000-configuration study showing P0 wins 67.2% with zero draws, a fragile advantage (mean 1.37 winning openings), and near-invariant state-space size (std/mean = 2.2%).
3. **Win-dynamics analysis:** identification of the turn-parity sawtooth in $f(L)$, the stuck-rule dominance at late levels (76% at L=16), and the shrinking critical move ratio quantifying strategic crystallisation.
4. **Heuristic distillation:** extraction of interpretable playing rules from the complete solution, showing that a 4-rule heuristic achieves 65% move accuracy and a depth-4 decision tree reaches 74% (depth-8: 78%), while the opening (L=0) resists all feature-based approximation—confirming the fragile first-mover advantage is rooted in global board structure.

Open questions. Several directions remain:

- **GNN outcome prediction:** can a graph neural network on the tile-relation graph predict the game value without solving?
- **Full enumeration:** solving all $\sim 4.5 \times 10^9$ distinct configurations ($4.5 \times 10^9 \times 3.5\,\text{s} \approx$ 500 GPU-years on a single RTX 4090; tractable with large-scale parallelism).
- **Human play study:** the heuristic hierarchy (Section 7) brackets human-playable performance between 41% (random) and 78% (learned tree) move accuracy. Empirical studies with human participants would locate actual play on this spectrum and test whether the depth-4 rules improve human performance.

- **Multi-configuration heuristic validation:** do the MI profiles and move-accuracy results from the canonical board generalise across all 1,000 solved configurations?
- $N \times N$ **complexity:** extending Okiya's mechanics to larger grids—what are the complexity thresholds for tractability?

A Win Pattern Bitmasks

The 19 winning patterns are encoded as 16-bit bitmasks over the 4×4 board (row-major order, position p corresponds to bit 2^p). A player wins if $(mask \,\&\, pattern) = pattern$ for any pattern.

Table 7: Win pattern bitmasks.

Category	Hex	Decimal
Row 0	`0xF000`	61440
Row 1	`0x0F00`	3840
Row 2	`0x00F0`	240
Row 3	`0x000F`	15
Column 0	`0x8888`	34952
Column 1	`0x4444`	17476
Column 2	`0x2222`	8738
Column 3	`0x1111`	4369
Diagonal \	`0x8421`	33825
Diagonal /	`0x1248`	4680
Block (0,0)	`0xCC00`	52224
Block (0,1)	`0x6600`	26112
Block (0,2)	`0x3300`	13056
Block (1,0)	`0x0CC0`	3264
Block (1,1)	`0x0660`	1632
Block (1,2)	`0x0330`	816
Block (2,0)	`0x00CC`	204
Block (2,1)	`0x0066`	102
Block (2,2)	`0x0033`	51

B Notation

Code Availability

Source code, solver, batch-analysis scripts, and all data used in this paper are available at https://github.com/format37/okiya.

References

[1] E. Zermelo, "Über eine Anwendung der Mengenlehre auf die Theorie des Schachspiels," in *Proc. Fifth Int. Congress of Mathematicians*, vol. 2, pp. 501–504, 1913.

[2] J. von Neumann, "Zur Theorie der Gesellschaftsspiele," *Mathematische Annalen*, vol. 100, no. 1, pp. 295–320, 1928.

Table 8: Notation reference.

Symbol	Meaning
P0, P1	Player 0 (Red, first mover), Player 1 (Blue)
L	Level: total number of tiles placed
$mask_{P0}$, $mask_{P1}$	16-bit bitmasks of claimed positions
$last\_pos$	Position of most recently placed tile (0–15, or 16=none)
$v(s)$	Minimax value of state s: $+1$ or -1
$f(L)$	P0-win fraction at level L (Eq. 2)
$\Delta f(L)$	First difference $f(L) - f(L{-}1)$
CMR	Critical move ratio
CSR	Compressed Sparse Row (graph storage format)
DAG	Directed acyclic graph
BFS	Breadth-first search

[3] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[4] K. Thompson, "Retrograde analysis of certain endgames," *ICCA Journal*, vol. 9, no. 3, pp. 131–139, 1986.

[5] V. Allis, "A knowledge-based approach of Connect-Four," Master's thesis, Vrije Universiteit Amsterdam, 1988.

[6] R. Gasser, "Solving Nine Men's Morris," *Computational Intelligence*, vol. 12, no. 1, pp. 24–41, 1996.

[7] H. J. van den Herik, J. W. H. M. Uiterwijk, and J. van Rijswijck, "Games solved: Now and in the future," *Artificial Intelligence*, vol. 134, no. 1–2, pp. 277–311, 2002.

[8] J. Schaeffer, N. Burch, Y. Björnsson, A. Kishimoto, M. Müller, R. Lake, P. Lu, and S. Sutphen, "Checkers is solved," *Science*, vol. 317, no. 5844, pp. 1518–1522, 2007.

[9] S. Edelkamp and D. Sulewski, "Efficient explicit-state model checking on general purpose graphics processors," in *Proc. SPIN*, LNCS 6349, pp. 106–123, 2010.

[10] A. Klöckner, N. Pinto, Y. Lee, B. Catanzaro, P. Ivanov, and A. Fasih, "PyCUDA and PyOpenCL: A scripting-based approach to GPU run-time code generation," *Parallel Computing*, vol. 38, no. 3, pp. 157–174, 2012.

[11] B. Cathala, *Okiya / Niya*, Blue Orange Games, 2013. BoardGameGeek ID: 125311.

[12] G. Irving, J. Donkers, and J. Uiterwijk, "Solving Pentago," Technical report, 2014.

[13] E. R. Berlekamp, J. H. Conway, and R. K. Guy, *Winning Ways for Your Mathematical Plays*, 2nd ed. A K Peters/CRC Press, 2018.

[14] M. J. H. Heule and O. Kullmann, "The science of brute force," *Communications of the ACM*, vol. 60, no. 8, pp. 70–79, 2017.